

Real-world (compare your loop to the library's) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below reuse `get_weather` and `celsius_to_fahrenheit` from `build_react_loop`, and assume `run_agent()` from that exercise's Solution is already in scope.

Basic Version

```
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

@tool
def get_weather_tool(city: str) -> str:
    """Get the current weather for a city, in Celsius."""
    return get_weather(city)

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])
agent = create_tool_calling_agent(llm, [get_weather_tool], prompt)
executor = AgentExecutor(agent=agent, tools=[get_weather_tool], max_iterations=5,

result = executor.invoke({"input": "What's the weather in Paris?"})
print(result["output"])
```

Expected output (final line; `verbose=True` also prints the Thought/Action/Observation steps above it):

```
It's 18°C and cloudy in Paris right now.
```


[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — 3 test questions, both loops, diffed

```
test_questions = [
    "What's the weather in Paris?",
    "What's the weather in Paris, in Fahrenheit?",
    "What's the capital of France?",
]

for question in test_questions:
    mine = run_agent(question)
    lib = executor.invoke({"input": question})

    print("QUESTION:", question)
    print("  mine:", mine.final_answer, "|", len(mine.steps), "tool call(s)")
    print("  lib: ", lib["output"])
    print()
```

Expected output (shape; exact wording varies run to run):

```
QUESTION: What's the weather in Paris?
mine: It's 18°C and cloudy in Paris right now. | 1 tool call(s)
lib:  It's 18°C and cloudy in Paris right now.

QUESTION: What's the weather in Paris, in Fahrenheit?
mine: It's 64.4°F in Paris. | 2 tool call(s)
lib:  It's 64.4°F in Paris right now.

QUESTION: What's the capital of France?
mine: The capital of France is Paris. | 0 tool call(s)
lib:  The capital of France is Paris.
```

Approach 2 — counting `AgentExecutor` 's steps directly, not just trusting `verbose`

```
result = executor.invoke({"input": "What's the weather in Paris, in Fahrenheit?"})
print(result["output"])
print("lib step count:", len(result["intermediate_steps"]))
```

`result["intermediate_steps"]` is a list of `(action, observation)` pairs — the library's own equivalent of your `steps` list — so you can compare step *counts* precisely instead of eyeballing the `verbose=True` printout. **Expected output:**

```
It's 64.4°F in Paris right now.  
lib step count: 2
```

Difference from Basic: Basic runs one question through the library loop alone. Approach 1 runs the *same* 3 questions through both loops and prints them side by side, so you can actually compare, not just observe each one in isolation. Approach 2 replaces "reading the `verbose=True` printout" with reading `result["intermediate_steps"]` directly — the more reliable way to compare step counts, since `verbose` output is meant for humans to read, not for code to parse.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a failing tool, compared side by side

```
call_count = {"n": 0}

@tool
def flaky_weather_tool(city: str) -> str:
    """Get the current weather for a city. Fails on the first call."""
    call_count["n"] += 1
    if call_count["n"] == 1:
        raise RuntimeError("weather service timed out")
    return get_weather(city)

flaky_agent = create_tool_calling_agent(llm, [flaky_weather_tool], prompt)
flaky_executor = AgentExecutor(
    agent=flaky_agent, tools=[flaky_weather_tool], max_iterations=5, verbose=True
)
result = flaky_executor.invoke({"input": "What's the weather in Paris?"})
print(result["output"])
```

Expected behavior: `AgentExecutor` catches the `RuntimeError` on its own and feeds a message like `"Invalid or incomplete response"` (or the exception text, depending on version) back to the model as the `Observation` — the model then typically retries, and the second call succeeds. Your own hand-built `run_tool()` from `build_react_loop`'s Advanced Version does the same *shape* of thing (catch, feed back as `Observation`, let the model react) — but it feeds back the exact error message you chose (`f"Error: {e}"`), not a library-chosen generic one. This is the concrete answer to Hint 1 Advanced's question: both recover, but the observation text the model sees is different, and that difference can change what the model does next.

Approach 2 — hitting the step limit, compared side by side

```
short_executor = AgentExecutor(agent=agent, tools=[get_weather_tool], max_iterati

try:
    result = short_executor.invoke({"input": "What's the weather in Paris, in Fah
    print("lib result at limit:", result["output"])
except Exception as e:
```

```

print("lib raised:", e)

try:
    run_agent("What's the weather in Paris, in Fahrenheit?", max_iterations=1)
except MaxIterationsExceeded as e:
    print("mine raised:", e)

```

Expected output:

```

lib result at limit: Agent stopped due to iteration limit or time limit.
mine raised: No answer after 1 steps

```

The real difference: your `run_agent()` raises `MaxIterationsExceeded` — the Build Task requires this ("raised, not silently ignored"). `AgentExecutor`'s default behavior is to *return* a stopped-early message as if it were a normal answer, not raise — so code calling `executor.invoke(...)` that only checks `result["output"]` for truthiness would treat a step-limit failure as if it were a real, successful answer. This is worth knowing before you rely on `AgentExecutor` anywhere that needs to distinguish "answered" from "gave up."

Difference from Intermediate: Intermediate compares final answers and step counts on the happy path, where both loops behave almost identically. Advanced compares the 2 places they diverge — a tool failure, and the step limit — and finds a real, practical difference in each: the exact error text the model sees, and whether hitting the limit raises or just returns quietly.

Which one should you actually use? For Project 2 and most real work: `AgentExecutor` (or `LangGraph`, covered later) — it's tested, handles edge cases you'd otherwise reinvent, and having built `run_agent()` yourself means you now know what it's doing well enough to trust it, and precisely enough to know where to look when it does something unexpected. Keep your own hand-built loop only as a reference for exactly this kind of comparison, or for a case needing behavior `AgentExecutor` genuinely doesn't support — like raising loudly instead of returning quietly on a step-limit hit, which Advanced Approach 2 shows you'd need to add yourself (checking `result["output"]` against `AgentExecutor`'s exact stop message, or wrapping the call to raise on it).